

Conflict Resolution Strategy: Version Synchronization

The merge conflict encountered in this assignment was a "content conflict" occurring on the version metadata line of `main.py`. This happened because the `feature/multiply-function` and `feature/divide-function` branches diverged from the same base commit (Version 1.0.0) and independently modified the exact same lines of code.

When merging the `multiply` branch into `main`, Git performed a straightforward update. However, the subsequent attempt to merge the `divide` branch failed because Git identified "Divergent History"; it could not automatically determine whether the version should be **1.1.0** (from the first merge) or **1.0.1** (from the incoming branch).

To resolve this, I employed a **manual intervention strategy** using the VS Code Merge Editor. The resolution process involved:

1. **Logic Preservation:** Accepting incoming changes from both branches to ensure both the `multiply` and `divide` function implementations were kept in the final source code.
2. **Semantic Versioning:** Manually overriding both conflicting version strings to **1.2.0**. This reflects the successful integration of two new features into the project's main line.
3. **Functional Verification:** Consolidating the test block to ensure all mathematical operations executed correctly in the final build.

This approach ensured code integrity while maintaining a clear, linear progression of the project's version history.